INGENIERÍA DEL SOFTWARE II

- Tema 2. Procesos de desarrollo de software (Parte 3) -

Grado en Ingeniería Informática - Curso 2026/2027

Rocío Carratalá Sáez (rocio.carratala@uv.es)

Contenido

1. Introducción a los procesos de desarrollo
2. MÉTRICA versión 3
3. Metodologías ágiles
4. DevOps: continuidad entre desarrollo y operación

Programación extrema

La Programación Extrema (XP): metodología ágil de desarrollo de software que busca mejorar la calidad del producto y la capacidad de adaptación a los cambios.

- Se apoya en comunicación, simplicidad, retroalimentación, coraje y respeto.
- Sus prácticas cubren la relación con el cliente, la selección del trabajo, la construcción del software y el ritmo del equipo.
- A diferencia de los enfoques tradicionales, se centra en el cliente y sugiere ciclos de desarrollo cortos para entregar software funcional de manera continua y rápida; esto permite incorporar cambios de requisitos en cualquier fase del proyecto.

Programación extrema

¿En qué se basa?

- **Juego de planificación e historias:** cliente y equipo conversan sobre valor, alcance, riesgo y coste. Las historias expresan capacidades negociables; el cliente ordena y el equipo estima desde su conocimiento técnico.

Programación extrema

¿En qué se basa?

- **Juego de planificación e historias.**
- **Equipo completo y cliente disponibles:** las decisiones requieren acceso frecuente a personas capaces de aclarar necesidades, aportar ejemplos y resolver prioridades.

Programación extrema

¿En qué se basa?

- **Juego de planificación e historias:**
- **Equipo completo y cliente disponibles**.
- **Entregas pequeñas:** el producto se entrega en incrementos reducidos que conservan valor y calidad.

Programación extrema

¿En qué se basa?

- **Juego de planificación e historias.**
- **Equipo completo y cliente disponibles**.
- **Entregas pequeñas**.
- **Pruebas de aceptación:** cliente y equipo concretan ejemplos observables para una historia. Estas pruebas orientan la conversación, complementan los criterios de aceptación y permiten comprobar que el incremento satisface el comportamiento acordado.

Programación extrema

¿En qué se basa?

- **Juego de planificación e historias.**
- **Equipo completo y cliente disponible**.
- **Entregas pequeñas**.
- **Pruebas de aceptación**.
- **Metáfora y vocabulario compartido:** una imagen sencilla o un modelo común ayuda a explicar la arquitectura y las responsabilidades. Debe aclarar el sistema sin sustituir modelos precisos cuando existen reglas, estados o integraciones complejas.

Programación extrema

Prácticas técnicas que sustentan la XP (capaces de cambios frecuentes):

- **Desarrollo guiado por pruebas**. El desarrollo guiado por pruebas (TDD, Test-Driven Development) sigue un ciclo breve: escribir una prueba que falla, implementar lo necesario para superarla y refactorizar. Favorece diseño incremental y protección frente a regresiones.

Programación extrema

Prácticas técnicas que sustentan la XP (capaces de cambios frecuentes):

- **Desarrollo guiado por pruebas**.
- **Programación por parejas**. Dos personas trabajan sobre el mismo cambio, alternan conducción y revisión y mantienen una conversación continua sobre diseño y riesgos. La programación en grupo o *ensemble programming* extiende la colaboración simultánea a más participantes.

Programación extrema

Prácticas técnicas que sustentan la XP (capaces de cambios frecuentes):

- **Desarrollo guiado por pruebas**.
- **Programación por parejas**.
- **Integración continua**. El equipo integra cambios pequeños con frecuencia en una línea compartida. Una construcción reproducible y una verificación rápida detectan incompatibilidades; el equipo repara de inmediato una línea rota.

Programación extrema

Prácticas técnicas que sustentan la XP (capaces de cambios frecuentes):

- **Desarrollo guiado por pruebas**.
- **Programación por parejas**.
- **Integración continua**.
- **Refactorización**. El diseño interno mejora mediante cambios pequeños que preservan el comportamiento observable. Las pruebas, el control de versiones y la integración frecuente reducen el riesgo de cada transformación.

Programación extrema

Prácticas técnicas que sustentan la XP (capaces de cambios frecuentes):

- **Desarrollo guiado por pruebas**.
- **Programación por parejas**.
- **Integración continua**.
- **Refactorización**.
- **Diseño simple**. El diseño satisface las necesidades y propiedades de calidad conocidas con la menor complejidad suficiente. Pasa las pruebas, comunica intención, evita duplicación relevante y prescinde de elementos sin función actual.

Programación extrema

Prácticas técnicas que sustentan la XP (capaces de cambios frecuentes):

- **Desarrollo guiado por pruebas**.
- **Programación por parejas**.
- **Integración continua**.
- **Refactorización**.
- **Diseño simple**.
- **Propiedad colectiva del código**. Cualquier integrante preparado puede mejorar cualquier parte del código. El conocimiento y la responsabilidad se distribuyen, de modo que un módulo no depende de una única persona.

Programación extrema

Prácticas técnicas que sustentan la XP (capaces de cambios frecuentes):

- **Desarrollo guiado por pruebas**.
- **Programación por parejas**.
- **Integración continua**.
- **Refactorización**.
- **Diseño simple**.
- **Propiedad colectiva del código**.
- **Estándares de codificación**. El equipo comparte convenciones de nombres, estructura, formato y diseño. Los estándares reducen fricción y permiten ejercer la propiedad colectiva sin degradar coherencia.

Programación extrema

Prácticas técnicas que sustentan la XP (capaces de cambios frecuentes):

- **Desarrollo guiado por pruebas**.
- **Programación por parejas**.
- **Integración continua**.
- **Refactorización**.
- **Diseño simple**.
- **Propiedad colectiva del código**.
- **Estándares de codificación**.
- **Ritmo sostenible**. El equipo mantiene durante el tiempo una carga compatible con calidad, aprendizaje y atención. La urgencia permanente aumenta defectos, retrabajo y dependencia de conocimiento tácito.

Programación extrema: tres principios relacionados

Inversión inicial

KISS. *Keep It Simple, Stupid*
Propone evitar complejidad innecesaria y elegir la solución más sencilla que satisfaga el problema actual. Sencillez requiere claridad, pruebas y comprensión del dominio; una solución incompleta o frágil solo desplaza el coste.

YAGNI. *You Aren't Gonna Need It*
Recomienda posponer capacidades especulativas hasta disponer de una necesidad y evidencia suficientes. Reduce código, opciones y mantenimiento anticipados, mientras las decisiones difíciles de revertir siguen analizándose con antelación.

Controlar coste de cambio

DRY. *Don't Repeat Yourself*
Busca que cada conocimiento relevante tenga una representación autoritativa.
La duplicación de una regla de negocio puede producir respuestas contradictorias; abstraer fragmentos parecidos sin una razón común también aumenta acoplamiento.

Deuda técnica

Es el **coste y el riesgo futuro que genera una decisión técnica actual** al hacer que el software resulte más difícil de comprender, modificar, probar, desplegar u operar.

Puede asumirse deliberadamente para entregar antes o aprender, **pero debe registrarse** indicando:

- qué decisión la originó;
- qué consecuencias produce;
- qué partes del sistema afecta;
- cuándo conviene corregirla;
- qué ocurrirá si continúa acumulándose.

Como una deuda financiera, puede generar "intereses": cada cambio posterior requiere más tiempo, aumenta el riesgo de defectos y reduce la capacidad del equipo para responder. **No todo código imperfecto constituye deuda técnica; debe existir una consecuencia sobre el desarrollo o mantenimiento futuro.**

Deuda técnica

Fuente: https://jrgsanta.com



Es el **coste y el riesgo** **ca actual** al hacer que el software resulte má ar, desplegar u operar.

Puede asumirse delib ender, **pero debe registrarse** indicando:

- qué decisión la origi
- qué consecuencias
- qué partes del siste
- cuándo conviene co
- qué ocurrirá si contin

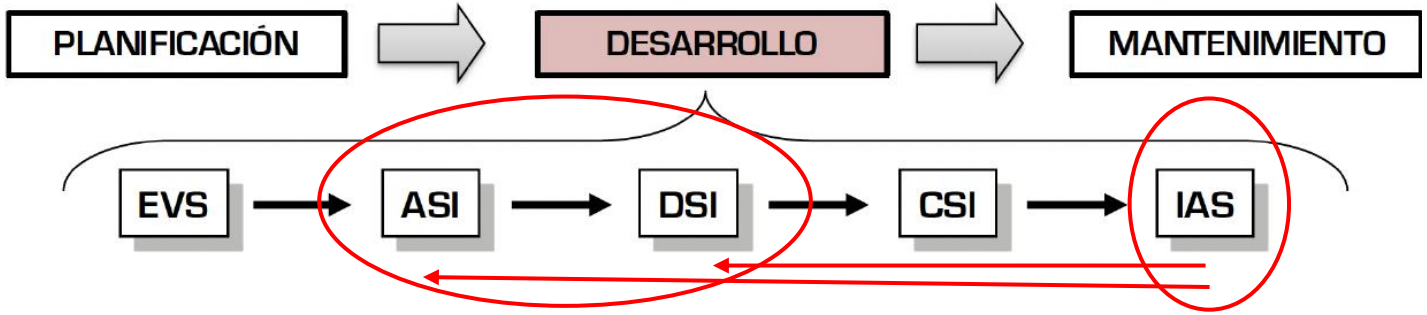
Como una deuda finan cambio posterior requiere más tiempo, a la capacidad del equipo para responder. **No todo código imperfecto constituye deuda técnica; debe existir una consecuencia sobre el desarrollo o mantenimiento futuro.**

2.4 DevOps: continuidad desarrollo-operación

DevOps **organiza capacidades y responsabilidades para entregar y operar software con retroalimentación rápida y fiable**.

Combina cultura de colaboración, prácticas técnicas, automatización y medición, y abarca el flujo compartido de desarrollo y operaciones.

Conserva las **responsabilidades** de IAS (Implantación y Aceptación de Sistemas), automatiza y repite muchas de sus actividades, acerca operación al diseño y **devuelve evidencia de producción** a ASI (Análisis del Sistema de Información) y DSI (Desarrollo de Sistemas de Información).



Flujo de entrega moderno de código

construir → verificar → proteger → empaquetar → desplegar → observar → aprender

Cada flecha debe ser rápida, repetible y trazable.

La **automatización** reduce variación y coste de repetición, pero conserva decisiones humanas donde existe juicio: aceptar un riesgo, interpretar impacto o decidir una reversión.

Flujo de entrega moderno de código

construir → verificar → proteger → empaquetar → desplegar → observar → aprender

Prácticas que sustentan este flujo:

- **control de versiones** para código, configuración, infraestructura y documentación operativa;

Flujo de entrega moderno de código

construir → verificar → proteger → empaquetar → desplegar → observar → aprender

Prácticas que sustentan este flujo:

- **control de versiones** para código, configuración, infraestructura y documentación operativa;
- **integración continua** para integrar cambios pequeños y verificarlos pronto;

Flujo de entrega moderno de código

construir → verificar → proteger → empaquetar → desplegar → observar → aprender

Prácticas que sustentan este flujo:

- **control de versiones** para código, configuración, infraestructura y documentación operativa;
- **integración continua** para integrar cambios pequeños y verificarlos pronto;
- **entrega continua** para mantener el producto en estado desplegable;

Flujo de entrega moderno de código

construir → verificar → proteger → empaquetar → desplegar → observar → aprender

Prácticas que sustentan este flujo:

- **control de versiones** para código, configuración, infraestructura y documentación operativa;
- **integración continua** para integrar cambios pequeños y verificarlos pronto;
- **entrega continua** para mantener el producto en estado desplegable;
- **despliegue progresivo** para limitar exposición y aprender con seguridad;

Flujo de entrega moderno de código

construir → verificar → proteger → empaquetar → desplegar → observar → aprender

Prácticas que sustentan este flujo:

- **control de versiones** para código, configuración, infraestructura y documentación operativa;
- **integración continua** para integrar cambios pequeños y verificarlos pronto;
- **entrega continua** para mantener el producto en estado desplegable;
- **despliegue progresivo** para limitar exposición y aprender con seguridad;
- **observabilidad** para comprender el estado interno mediante señales del sistema;

Flujo de entrega moderno de código

construir → verificar → proteger → empaquetar → desplegar → observar → aprender

Prácticas que sustentan este flujo:

- **control de versiones** para código, configuración, infraestructura y documentación operativa;
- **integración continua** para integrar cambios pequeños y verificarlos pronto;
- **entrega continua** para mantener el producto en estado desplegable;
- **despliegue progresivo** para limitar exposición y aprender con seguridad;
- **observabilidad** para comprender el estado interno mediante señales del sistema;
- **respuesta y aprendizaje de incidentes** sin ocultar factores sistémicos;

Flujo de entrega moderno de código

construir → verificar → proteger → empaquetar → desplegar → observar → aprender

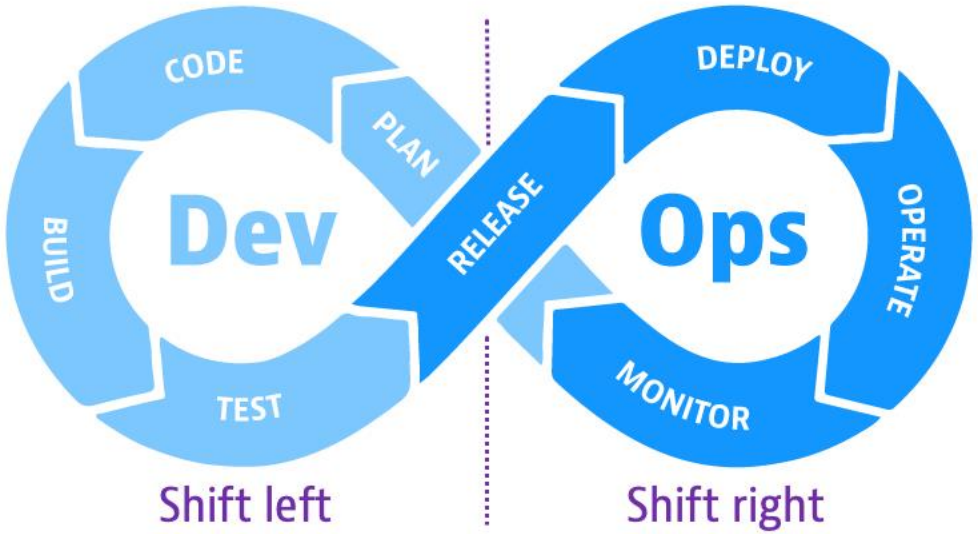
Prácticas que sustentan este flujo:

- **control de versiones** para código, configuración, infraestructura y documentación operativa;
- **integración continua** para integrar cambios pequeños y verificarlos pronto;
- **entrega continua** para mantener el producto en estado desplegable;
- **despliegue progresivo** para limitar exposición y aprender con seguridad;
- **observabilidad** para comprender el estado interno mediante señales del sistema;
- **respuesta y aprendizaje de incidentes** sin ocultar factores sistémicos;
- **plataformas y autoservicio** para ofrecer caminos seguros sin crear nuevos silos.

Entrega moderna de código: controles en el flujo

Shift left y **shift right** son dos estrategias de ingeniería de software que mueven las pruebas, la seguridad y el control de calidad hacia los **extremos del ciclo de vida de desarrollo** (SDLC), visualizado como una línea de izquierda a derecha.

shift left
previene fallos
antes de lanzar el producto



shift right
asegura la estabilidad y mejora continua del **producto ya lanzado**

Fuente: https://news.cdn.dm.dynatrace.com/

Entrega moderna de código: controles en el flujo

Shift Left (Mover a la izquierda) significa integrar las pruebas, la seguridad y la calidad en las fases iniciales del desarrollo (planificación, diseño y codificación).

OBJETIVO: detectar y corregir errores, fallos de seguridad o problemas de requisitos antes de que el código avance.

VENTAJA: reduce drásticamente los costos de reparación, acelera las entregas y evita que los defectos crezcan.

¿QUÉ USA? Pruebas unitarias hechas por desarrolladores, análisis estático de código (SAST) y revisión temprana de requisitos.

Entrega moderna de código: controles en el flujo

Shift Right (Mover a la derecha) significa extender el control de calidad y las pruebas hacia la etapa de producción, una vez que el software ya está en manos de los usuarios.

OBJETIVO: monitorear el comportamiento real, el rendimiento, el uso y la seguridad del sistema en entornos reales.

VENTAJA: permite aprender de la experiencia del usuario y detectar fallos que no aparecieron en las pruebas previas.

¿QUÉ USA?

- Despliegues *canary* (lanzar nuevo software en producción de forma gradual, enviando la nueva versión solo a un pequeño grupo de usuarios o servidores antes de dársela a todo el mundo),
- Pruebas A/B (experimentos que comparan dos versiones de un elemento para ver cuál funciona mejor en función de métricas clave),
- Monitorización continua y simulaciones de fallo (*chaos engineering*)

Métricas de entrega: DORA

La iniciativa **DevOps Research and Assessment (DORA)** agrupa actualmente **cinco métricas en rendimiento de flujo e inestabilidad**:

Métrica	Lectura	Uso prudente
Tiempo de entrega del cambio	Desde el commit hasta el despliegue en producción.	Localizar esperas y reducir el tamaño del lote.
Frecuencia de despliegue	Número de despliegues por periodo o intervalo entre ellos.	Observar la capacidad de entregar y su evolución en el servicio.
Tiempo de recuperación de despliegue fallido	Tiempo para recuperarse de un despliegue que exige intervención inmediata.	Mejorar detección, diagnóstico, reversión y respuesta.
Tasa de fallo de cambio	Proporción de despliegues que requieren intervención inmediata.	Estudiar estabilidad del proceso de entrega.
Tasa de retrabajo de despliegue	Proporción de despliegues no planificados derivados de incidentes.	Hacer visible capacidad consumida por corregir entregas previas.

Jerarquía de cargos en una empresa *TIC*

Dirección y gobierno

- CEO (Chief **Executive** Officer) - Resultado **global** de la empresa, estrategia y rendición de cuentas ante propietarios o consejo.
- CIO (Chief **Information** Officer) - Uso de la **información** y de los sistemas internos para sostener el negocio.
- CTO (Chief **Technology** Officer) - Capacidad **tecnológica** con la que se construyen y evolucionan productos y plataformas.
- CPO (Chief **Product** Officer) - **Estrategia** y **resultados** del conjunto de productos.
- CISO (Chief Information **Security** Officer) - Gobierno del riesgo de **seguridad** y protección de la información.

↓ objetivos, inversión, políticas y límites de riesgo ↓

Jerarquía de cargos en una empresa *TIC*

Dirección y gobierno: CEO · CIO · CTO · CPO · CISO

↓ objetivos, inversión, políticas y límites de riesgo ↓

Producto, entrega y equipos

- Product Manager - **Problema** de mercado o de usuario, **estrategia** del producto y **resultados a medio plazo**.
- Product Owner - Maximización del valor y gestión eficaz del **Product Backlog**.
- Project Manager - **Cumplimiento de un encargo dentro de restricciones** de alcance, plazo, coste, recursos y riesgo.
- Engineering Manager - **Salud y capacidad sostenible** de los equipos técnicos.
- (Scrum Master)

↓ prioridades, capacidad, coordinación y aprendizaje ↓

Jerarquía de cargos en una empresa *TIC*

Dirección y gobierno: CEO · CIO · CTO · CPO · CISO

↓ objetivos, inversión, políticas y límites de riesgo ↓

Producto, entrega y equipos: Product Manager · Product Owner · Project Manager · Engineering Manager · (Scrum Master)

↓ prioridades, capacidad, coordinación y aprendizaje ↓

Diseño, construcción y operación

Arquitecto/a de software · Tech Lead · Ingeniero/a o desarrollador/a de software

Ing. de calidad · Diseño de experiencia de usuario · Datos y admin. de BBDD

Plataforma y fiabilidad · Seguridad · Operación y soporte

Coherencia de la arquitectura entre sistemas
y decisiones con consecuencias amplias o duraderas.

Jerarquía de cargos en una empresa *TIC*

Dirección y gobierno: CEO · CIO · CTO · CPO · CISO

↓ objetivos, inversión, políticas y límites de riesgo ↓

Producto, entrega y equipos: Product Manager · Product Owner · Project Manager · Engineering Manager · (Scrum Master)

↓ prioridades, capacidad, coordinación y aprendizaje ↓

Diseño, construcción y operación

Arquitecto/a de software · Tech Lead · Ingeniero/a o desarrollador/a de software

Ing. de calidad · Diseño de experiencia de usuario · Datos y admin. de BBDD

Plataforma y fiabilidad · Seguridad · Operación y soporte

Dirección técnica próxima al trabajo cotidiano de un equipo.

Jerarquía de cargos en una empresa *TIC*

Dirección y gobierno: CEO · CIO · CTO · CPO · CISO

↓ objetivos, inversión, políticas y límites de riesgo ↓

Producto, entrega y equipos: Product Manager · Product Owner · Project Manager · Engineering Manager · (Scrum Master)

↓ prioridades, capacidad, coordinación y aprendizaje ↓

Diseño, construcción y operación

Arquitecto/a de software · Tech Lead · Ingeniero/a o desarrollador/a de software

Ing. de calidad · Diseño de experiencia de usuario · Datos y admin. de BBDD

Plataforma y fiabilidad · Seguridad · Operación y soporte

Construcción, prueba, mantenimiento y evolución del producto.

Jerarquía de cargos en una empresa *TIC*

Dirección y gobierno: CEO · CIO · CTO · CPO · CISO

↓ objetivos, inversión, políticas y límites de riesgo ↓

Producto, entrega y equipos: Product Manager · Product Owner · Project Manager · Engineering Manager · (Scrum Master)

↓ prioridades, capacidad, coordinación y aprendizaje ↓

Diseño, construcción y operación

Arquitecto/a de software · Tech Lead · Ingeniero/a o desarrollador/a de software

Ing. de calidad · Diseño de experiencia de usuario · Datos y admin. de BBDD

Plataforma y fiabilidad · Seguridad · Operación y soporte

Estrategia de calidad y obtención de evidencia sobre el comportamiento del sistema.

Jerarquía de cargos en una empresa *TIC*

Dirección y gobierno: CEO · CIO · CTO · CPO · CISO

↓ objetivos, inversión, políticas y límites de riesgo ↓

Producto, entrega y equipos: Product Manager · Product Owner · Project Manager · Engineering Manager · (Scrum Master)

↓ prioridades, capacidad, coordinación y aprendizaje ↓

Diseño, construcción y operación

Arquitecto/a de software · Tech Lead · Ingeniero/a o desarrollador/a de software

Ing. de calidad · Diseño de experiencia de usuario · Datos y admin. de BBDD

Plataforma y fiabilidad · Seguridad · Operación y soporte

Comprensión de usuarios, interacción, accesibilidad y coherencia de la experiencia.

Jerarquía de cargos en una empresa *TIC*

Dirección y gobierno: CEO · CIO · CTO · CPO · CISO

↓ objetivos, inversión, políticas y límites de riesgo ↓

Producto, entrega y equipos: Product Manager · Product Owner · Project Manager · Engineering Manager · (Scrum Master)

↓ prioridades, capacidad, coordinación y aprendizaje ↓

Diseño, construcción y operación

Arquitecto/a de software · Tech Lead · Ingeniero/a o desarrollador/a de software

Ing. de calidad · Diseño de experiencia de usuario · Datos y admin. de BBDD

Plataforma y fiabilidad · Seguridad · Operación y soporte

Calidad, disponibilidad, seguridad y evolución de datos y bases de datos.

Jerarquía de cargos en una empresa *TIC*

Dirección y gobierno: CEO · CIO · CTO · CPO · CISO

↓ objetivos, inversión, políticas y límites de riesgo ↓

Producto, entrega y equipos: Product Manager · Product Owner · Project Manager · Engineering Manager · (Scrum Master)

↓ prioridades, capacidad, coordinación y aprendizaje ↓

Diseño, construcción y operación

Arquitecto/a de software · Tech Lead · Ingeniero/a o desarrollador/a de software

Ing. de calidad · Diseño de experiencia de usuario · Datos y admin. de BBDD

Plataforma y fiabilidad · Seguridad · Operación y soporte

Entornos, automatización, observabilidad, capacidad y fiabilidad del servicio.

Jerarquía de cargos en una empresa *TIC*

Dirección y gobierno: CEO · CIO · CTO · CPO · CISO

↓ objetivos, inversión, políticas y límites de riesgo ↓

Producto, entrega y equipos: Product Manager · Product Owner · Project Manager · Engineering Manager · (Scrum Master)

↓ prioridades, capacidad, coordinación y aprendizaje ↓

Diseño, construcción y operación

Arquitecto/a de software · Tech Lead · Ingeniero/a o desarrollador/a de software

Ing. de calidad · Diseño de experiencia de usuario · Datos y admin. de BBDD

Plataforma y fiabilidad · Seguridad · Operación y soporte

Prevención, detección y tratamiento de riesgos de seguridad durante el ciclo de vida.

Jerarquía de cargos en una empresa *TIC*

Dirección y gobierno: CEO · CIO · CTO · CPO · CISO

↓ objetivos, inversión, políticas y límites de riesgo ↓

Producto, entrega y equipos: Product Manager · Product Owner · Project Manager · Engineering Manager · (Scrum Master)

↓ prioridades, capacidad, coordinación y aprendizaje ↓

Diseño, construcción y operación

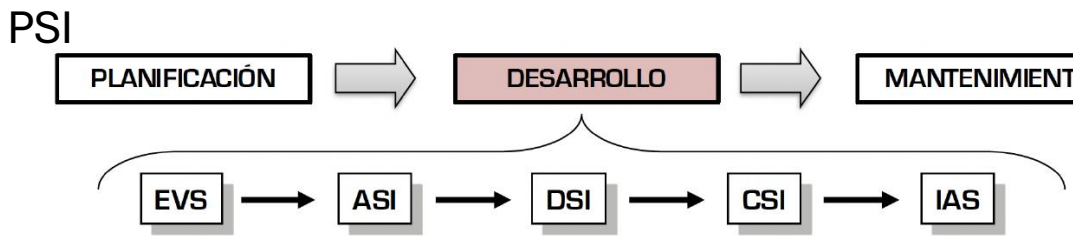
Arquitecto/a de software · Tech Lead · Ingeniero/a o desarrollador/a de software

Ing. de calidad · Diseño de experiencia de usuario · Datos y admin. de BBDD

Plataforma y fiabilidad · Seguridad · Operación y soporte

Continuidad del servicio y atención de incidencias y consultas.

Cargos - responsabilidades



VNIVERSITAT DE VALÈNCIA

Ámbito	Participación habitual	Relación con el tema
Estrategia y cartera	CEO, CIO, CPO, CTO, CISO y comités de dirección.	PSI y EVS convierten estrategia, necesidades, viabilidad y riesgo en decisiones de inversión.
Análisis y producto	Product Manager, Product Owner, usuarios, UX y especialistas del dominio.	ASI, Product Goal y Product Backlog concretan necesidades, requisitos, prioridades y resultados esperados.
Diseño y construcción	Arquitectura, Tech Lead, desarrollo, QA, datos y seguridad.	DSI y CSI producen decisiones técnicas, componentes integrados y evidencia de verificación.
Coordinación y mejora	Project Manager, Engineering Manager y Scrum Master.	La interfaz de gestión de proyectos, Scrum y Kanban hacen visibles compromisos, capacidad, flujo, impedimentos y riesgos.
Implantación y servicio	Plataforma, SRE, operación, soporte, seguridad y equipos de desarrollo.	IAS, MSI y DevOps conectan aceptación, despliegue, observación, mantenimiento y aprendizaje operativo.